WEEK 10

Networking: Retrofit, JSON & Images

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Almost every real app talks to a server: logging in, loading products, sending messages. This week you'll learn how apps communicate over the internet with **HTTP** and **JSON**, how to call APIs with **Retrofit**, how to handle loading and errors properly, and how to show images from the internet with **Coil**. You'll build a **Weather app** using a free API.

Day	Topic	You will be able to...
Day 1	How apps talk to servers	Explain HTTP requests, responses, status codes and JSON
Day 2	JSON with kotlinx.serialization	Turn JSON text into Kotlin objects and back
Day 3	Retrofit	Define an API interface and make network calls
Day 4	Repository, errors & UiState	Structure networking code like a team and handle failures
Day 5	Images and lists from the network	Show network images with Coil and send data with POST
Day 6	Weekly project	Build a Weather app with the Open-Meteo API

Tip: The JSON examples were **run** with the real kotlinx.serialization library, and the outputs are real. Retrofit, OkHttp and Coil couldn't be downloaded here, so that code was compile-checked against stand-ins that copy their APIs. The lessons use two free APIs that need no key: **Open-Meteo** (weather) and **JSONPlaceholder** (practice data). Test them in your browser first.

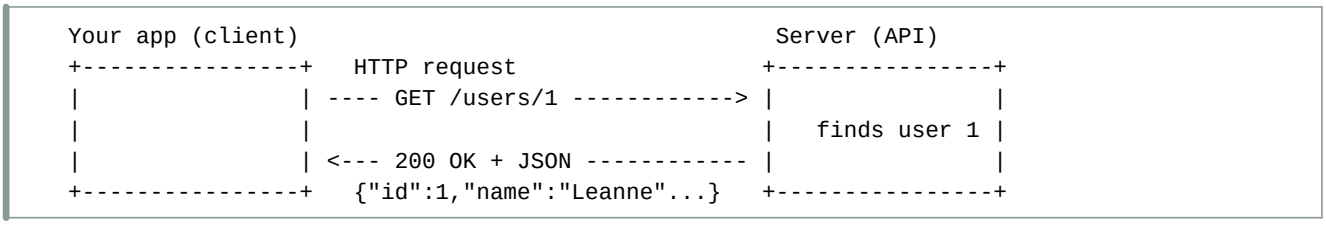
WEEK 10 • DAY 1

How Apps Talk to Servers

Today's goal: Understand HTTP requests and responses, URLs, methods, status codes and the JSON format.

1. Client and server

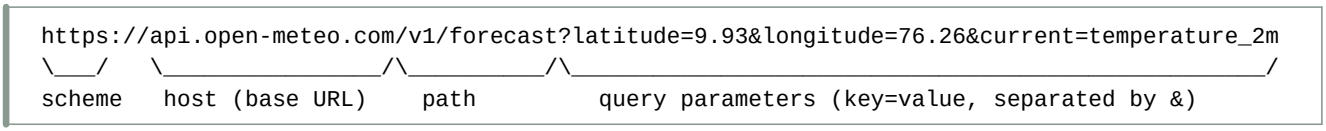
Sketch: A request and a response



The app sends a **request** ("give me user 1"). The server sends back a **response** with a **status code** (did it work?) and usually some **data** in JSON format.

2. Parts of a URL

Sketch: Anatomy of an API URL



3. HTTP methods

Method	Meaning	Example
GET	Read data	GET /posts: list of posts
POST	Create something new	POST /posts with a new post in the body
PUT / PATCH	Update something	PUT /posts/1: replace post 1
DELETE	Delete something	DELETE /posts/1

4. Status codes

Code	Meaning	What the app should do
200 OK / 201 Created	Success	Show the data
400 Bad Request	The app sent wrong data	Show a validation message
401 Unauthorized / 403 Forbidden	Not logged in / not allowed	Ask the user to log in again
404 Not Found	It doesn't exist	Show "not found"
500+ Server error	The server has a problem	Show "try again later" with a Retry button
(no response)	No internet or timeout	Show "check your connection"

5. JSON

JSON (JavaScript Object Notation) is the text format most APIs use. It looks a lot like Kotlin data classes:

A JSON response (JSONPlaceholder /users/1, shortened)

```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "city": "Gwenborough",
    "zipcode": "92998-3874"
  },
  "website": "hildegard.org"
}
```

JSON	Kotlin
{ ... } object	A data class
"name": "Leanne"	val name: String
"id": 1 / 1.5	val id: Int / Double
true / false	Boolean
[...] array	List<...>
null	A nullable type: String?

6. Try APIs in your browser

- Open **https://jsonplaceholder.typicode.com/users/1**: fake user data for practice.
- Open **https://jsonplaceholder.typicode.com/posts?userId=1**: the posts of user 1.
- Open **https://api.open-meteo.com/v1/forecast?latitude=9.93&longitude=76.26&current=temperature_2m**: the current temperature in Kochi.
- Install a JSON viewer extension in Chrome to make the responses easier to read.

7. Internet permission

AndroidManifest.xml (inside <manifest>, before <application>)

```
<uses-permission android:name="android.permission.INTERNET" />
```

Careful: Without this line, every network call fails with a SecurityException. It's the most common reason a beginner's first API call doesn't work.

Practice Exercises

1. Open the three URLs above in your browser and write down the names of 3 fields from each response.

2. Change the Open-Meteo URL to your own city's latitude and longitude (search "latitude of Thrissur").

3. For each situation, write the status code you'd expect: wrong password; a deleted post; the server crashed; a new account created.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which HTTP method reads data?

Q2. What does status code 404 mean?

Q3. What is the difference between 4xx and 5xx codes?

Q4. What does a JSON array become in Kotlin?

Q5. Which manifest permission do network calls need?

WEEK 10 • DAY 2

JSON with kotlinx.serialization

Today's goal: Convert JSON to Kotlin objects and back, including nested objects, lists, renamed and optional fields.

1. @Serializable data classes

Decoding JSON into an object

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class User(val id: Int, val name: String, val email: String)

fun main() {
    val json = """{"id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz"}"""

    val user = Json.decodeFromString<User>(json)
    println(user)
    println("Hello, ${user.name}!")
}
```

Output

```
User(id=1, name=Leanne Graham, email=Sincere@april.biz)
Hello, Leanne Graham!
```

Code	Meaning
@Serializable	Tells the Kotlin serialization plugin to generate the JSON conversion code for this class
Json.decodeFromString<User>(json)	Text → object (decoding, "parsing")
Json.encodeToString(user)	Object → text (encoding), for example to send data to a server

2. Encoding objects to JSON

Object to JSON

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
import kotlinx.serialization.encodeToString

@Serializable
data class NewPost(val userId: Int, val title: String, val body: String)

fun main() {
    val post = NewPost(1, "Learning Retrofit", "Today I made my first API call!")
    println(Json.encodeToString(post))
}
```

Output

```
{"userId":1,"title":"Learning Retrofit","body":"Today I made my first API call!"}
```

3. Nested objects and lists

Nested JSON

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class Address(val city: String, val zipcode: String)

@Serializable
data class User(val id: Int, val name: String, val address: Address)

fun main() {
    val json = """
        [
          {"id": 1, "name": "Leanne Graham", "address": {"city": "Gwenborough", "zipcode": "92998-3874"}},
          {"id": 2, "name": "Ervin Howell", "address": {"city": "Wisokyburgh", "zipcode": "90566-7771"}}
        ]
    """
    val users = Json.decodeFromString<List<User>>(json)
    for (u in users) {
        println("${u.name} lives in ${u.address.city}")
    }
}
```

Output

```
Leanne Graham lives in Gwenborough
Ervin Howell lives in Wisokyburgh
```

4. Real APIs send extra fields: ignoreUnknownKeys

Ignoring fields you don't need

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class User(val id: Int, val name: String)

val json = Json { ignoreUnknownKeys = true }

fun main() {
    val response = """{"id": 1, "name": "Leanne Graham", "username": "Bret", "phone": "1-770-736-8031"}"""
    println(json.decodeFromString<User>(response))

    try {
        Json.decodeFromString<User>(response)          // the default Json is strict
    } catch (e: Exception) {
        println("Strict Json failed: unknown key")
    }
}
```

Output

```
User(id=1, name=Leanne Graham)
Strict Json failed: unknown key
```

Tip: Always configure `Json { ignoreUnknownKeys = true }` for real APIs. Servers add new fields all the time, and you don't want your app to crash when they do.

5. Renamed, optional and nullable fields

@SerialName, defaults and nulls

```
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class CurrentWeather(
    @SerialName("temperature_2m") val temperature: Double,     // JSON name -> Kotlin name
    @SerialName("wind_speed_10m") val windSpeed: Double,
    @SerialName("weather_code") val weatherCode: Int = 0,      // default if missing
    val note: String? = null                                     // may be missing or null
)

val json = Json { ignoreUnknownKeys = true }

fun main() {
    val a = json.decodeFromString<CurrentWeather>(
        """{"time": "2026-10-16T12:00", "temperature_2m": 30.2, "wind_speed_10m": 9.4, "weather_code": 2}""")
    val b = json.decodeFromString<CurrentWeather>("""{"temperature_2m": 24.0, "wind_speed_10m": 3.1}""")
    println(a)
    println(b)
}
```

Output

```
CurrentWeather(temperature=30.2, windSpeed=9.4, weatherCode=2, note=null)
CurrentWeather(temperature=24.0, windSpeed=3.1, weatherCode=0, note=null)
```

Situation	Solution
JSON uses snake_case (temperature_2m)	@SerialName("temperature_2m") val temperature
A field is sometimes missing	Give it a default value
A field can be null	Make it nullable: String? = null
The JSON has fields you don't need	Json { ignoreUnknownKeys = true }

Practice Exercises

1. Decode this into a data class Post and print its title: {"userId": 1, "id": 7, "title": "magnam facilis", "body": "..."}

2. Decode a JSON array of 3 posts into List<Post> and print how many posts each userId has (groupBy, Week 2).

3. Create a @Serializable data class Todo(val userId: Int, val title: String, val completed: Boolean = false) and encode one to JSON.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does @Serializable do?

Q2. Which function turns JSON text into an object?

Q3. Why use ignoreUnknownKeys = true?

Q4. How do you map temperature_2m to a property called temperature?

Q5. How do you handle a field that is sometimes missing?

WEEK 10 • DAY 3

Retrofit

Today's goal: Add Retrofit to a project, describe an API as a Kotlin interface, and make network calls from a coroutine.

1. What is Retrofit?

Retrofit is the most popular networking library for Android. You describe the API as a Kotlin **interface**, and Retrofit writes all the networking code for you: building URLs, sending requests and converting JSON into your data classes.

2. Dependencies

app/build.gradle.kts (Android only, versions as in your project)

```
plugins {
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-kotlinx-serialization:2.11.0")
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")   // to see requests in Logcat
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
}
```

Remember the **INTERNET permission** in AndroidManifest.xml (Day 1).

3. Describe the API as an interface

PostApi.kt

```
@Serializable
data class PostDto(val userId: Int, val id: Int, val title: String, val body: String)

interface PostApi {
    @GET("posts")
    suspend fun getPosts(): List<PostDto>

    @GET("posts/{id}")
    suspend fun getPost(@Path("id") id: Int): PostDto

    @GET("posts")
    suspend fun getPostsByUser(@Query("userId") userId: Int): List<PostDto>
}
```

Annotation	Resulting request (base URL https://jsonplaceholder.typicode.com/)
@GET("posts")	GET https://jsonplaceholder.typicode.com/posts
@GET("posts/{id}") + @Path("id") id = 7	GET .../posts/7
@GET("posts") + @Query("userId") userId = 1	GET .../posts?userId=1
suspend fun	The call runs in a coroutine, and Retrofit does the work on a background thread
return type List<PostDto>	Retrofit decodes the JSON into this type for you

DTO means **Data Transfer Object**: a class that exactly matches the server's JSON. On Day 4 you'll convert DTOs into your app's own classes.

4. Create the Retrofit instance

NetworkModule.kt

```
@Serializable
data class PostDto(val userId: Int, val id: Int, val title: String, val body: String)

interface PostApi {
    @GET("posts")
    suspend fun getPosts(): List<PostDto>
}

object NetworkModule {
    private const val BASE_URL = "https://jsonplaceholder.typicode.com/"   // must end with /

    private val json = Json { ignoreUnknownKeys = true }

    private val okHttpClient = OkHttpClient.Builder()
        .addInterceptor(HttpLoggingInterceptor().apply { level = HttpLoggingInterceptor.Level.BODY })
        .connectTimeout(15, TimeUnit.SECONDS)
        .readTimeout(15, TimeUnit.SECONDS)
        .build()

    private val retrofit: Retrofit = Retrofit.Builder()
        .baseUrl(BASE_URL)
        .client(okHttpClient)
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()

    val postApi: PostApi = retrofit.create(PostApi::class.java)
}
```

Part	Job
BASE_URL	The start of every URL. **Must end with /**
OkHttpClient	The actual HTTP engine: timeouts, interceptors
HttpLoggingInterceptor (Level.BODY)	Prints every request and response in Logcat. Very useful for debugging (turn it off in release builds)
asConverterFactory(...)	Uses kotlinx.serialization to convert JSON
retrofit.create(PostApi::class.java)	Retrofit generates the implementation of your interface
object NetworkModule	One shared instance for the whole app (Week 3 object). With Hilt (Week 12), this moves into a Hilt module

5. Making a call

Calling the API from a ViewModel

```
@Serializable
data class PostDto(val userId: Int, val id: Int, val title: String, val body: String)

interface PostApi {
    @GET("posts")
    suspend fun getPosts(): List<PostDto>
}

class PostsViewModel(private val api: PostApi) : ViewModel() {
    private val _titles = MutableStateFlow<List<String>>(emptyList())
    val titles: StateFlow<List<String>> = _titles.asStateFlow()

    init {
        viewModelScope.launch {
            try {
                val posts = api.getPosts()                    // network call: suspends, doesn't block
                _titles.value = posts.take(10).map { it.title }
            } catch (e: IOException) {
                _titles.value = listOf("No internet connection")
            }
        }
    }
}
```

Tip: Look in Logcat for the tag **okhttp.OkHttpClient** to see the full request URL and the JSON response. When something doesn't work, this is the first place to check: is the URL right? What did the server answer?

6. Common Retrofit errors

Error	Cause and fix
SecurityException: Permission denied (missing INTERNET permission?)	Add the INTERNET permission to the manifest
IllegalArgumentException: baseUrl must end in /	Add the final / to BASE_URL
SerializationException: ... Encountered an unknown key	Use Json { ignoreUnknownKeys = true }
SerializationException: Field 'x' is required	The JSON doesn't have that field: add a default value or make it nullable
UnknownHostException	No internet, or a typo in the URL
NetworkOnMainThreadException	You called the network without suspend / a coroutine

Practice Exercises

1. Write a **UserApi** interface for JSONPlaceholder with getUsers() (GET users) and getUser(id) (GET users/{id}).
2. Add **getComments(postId)** for GET comments?postId=N using @Query.
3. Write a small ViewModel that loads the first 5 users' names into a StateFlow<List<String>>.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. How do you describe an API for Retrofit?

Q2. What is the difference between @Path and @Query?

Q3. What must the base URL end with?

Q4. What is a DTO?

Q5. Why add HttpLoggingInterceptor?

WEEK 10 • DAY 4

Repository, Errors & UiState

Today's goal: Put networking behind a repository, convert DTOs into app models, and handle every kind of failure in UiState.

1. Where networking code goes

Sketch: Layers with networking

```
 Screen  <-- UiState --  ViewModel  -- calls -->  Repository  -- calls -->  Retrofit API  -->  server
                          (state, logic)          (data source,            (PostApi,
                                                   mapping, errors)         DTOs)
```

Layer	Knows about	Doesn't know about
API interface	URLs, JSON, DTOs	Screens, UiState
Repository	The API (and later the database); converts DTOs to app models	Composables
ViewModel	The repository and UiState	Retrofit, JSON, URLs

2. DTO vs app model

Mapping a DTO to an app model (run with real JSON)

```
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class UserDto(
    val id: Int,
    val name: String,
    val email: String,
    val phone: String,
    val website: String,
    val company: CompanyDto
)

@Serializable
data class CompanyDto(val name: String)

data class User(val id: Int, val name: String, val email: String, val companyName: String)

fun UserDto.toUser() = User(id = id, name = name, email = email.lowercase(), companyName = company.name)

val json = Json { ignoreUnknownKeys = true }

fun main() {
    val response = """{"id": 1, "name": "Leanne Graham", "username": "Bret", "email": "Sincere@april.biz",
        "phone": "1-770-736-8031 x56442", "website": "hildegard.org",
        "company": {"name": "Romaguera-Crona", "catchPhrase": "Multi-layered client-server neural-net"}}"""
    val dto = json.decodeFromString<UserDto>(response)
    val user = dto.toUser()
    println(user)
}
```

Output

```
User(id=1, name=Leanne Graham, email=sincere@april.biz, companyName=Romaguera-Crona)
```

Why two classes? The **DTO** follows the server, and changes when the API changes. The **app model** (User) has exactly what the app needs, in the shape the app wants. If the server renames a field, you fix one mapping function, not every screen.

3. The repository

UserRepository

```
interface UserApi {
    @GET("users")
    suspend fun getUsers(): List<UserDto>
}

class UserRepository(private val api: UserApi) {
    suspend fun getUsers(): List<User> = api.getUsers().map { it.toUser() }
}
```

4. Kinds of failure

Failure	Exception	Message for the user
No internet, timeout, server unreachable	IOException (e.g. UnknownHostException, SocketTimeoutException)	"No internet connection. Check your connection and try again."
Server answered with an error code (404, 500)	HttpException (code())	"Something went wrong on our side. Please try again later."
JSON doesn't match the DTO	SerializationException	Same as a server error (and fix the DTO!)

5. The ViewModel and UiState

UsersViewModel with complete error handling

```
sealed interface UsersUiState {
    data object Loading : UsersUiState
    data class Success(val users: List<User>) : UsersUiState
    data class Error(val message: String) : UsersUiState
}

class UsersViewModel(private val repository: UserRepository) : ViewModel() {
    private val _uiState = MutableStateFlow<UsersUiState>(UsersUiState.Loading)
    val uiState: StateFlow<UsersUiState> = _uiState.asStateFlow()

    init { loadUsers() }

    fun loadUsers() {
        viewModelScope.launch {
            _uiState.value = UsersUiState.Loading
            _uiState.value = try {
                UsersUiState.Success(repository.getUsers())
            } catch (e: IOException) {
                UsersUiState.Error("No internet connection. Check your connection and try again.")
            } catch (e: HttpException) {
                UsersUiState.Error("Server error (${e.code()}). Please try again later.")
            }
        }
    }
}
```

6. The screen

UsersScreen with loading, error, retry and content

```
@Composable
fun UsersScreen(uiState: UsersUiState, onRetry: () -> Unit) {
    when (uiState) {
        UsersUiState.Loading -> Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
        is UsersUiState.Error -> Column(
            Modifier.fillMaxSize().padding(32.dp),
            verticalArrangement = Arrangement.Center,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Icon(Icons.Default.Warning, contentDescription = null, modifier = Modifier.size(48.dp))
            Spacer(Modifier.height(12.dp))
            Text(uiState.message, textAlign = TextAlign.Center)
            Spacer(Modifier.height(12.dp))
            Button(onClick = onRetry) { Text("Retry") }
        }
        is UsersUiState.Success -> LazyColumn {
            items(uiState.users, key = { it.id }) { user ->
                ListItem(
                    headlineContent = { Text(user.name) },
                    supportingContent = { Text("${user.email} - ${user.companyName}") }
                )
                HorizontalDivider()
            }
        }
    }
}
```

ListItem is a Material 3 component for standard list rows with headline, supporting text and optional leading/trailing content.

> **Tip:** Test the error state on purpose: turn on **airplane mode** and open the screen. Then turn it off and tap Retry. Testers and reviewers always try this.

Practice Exercises

1. Write a **PostDto** and a **Post** app model (id, title with the first letter capitalised, and a 100-character preview of body), plus a toPost() mapping. Test it on sample JSON.

2. Write a **PostRepository** and **PostsViewModel** with a sealed UiState, handling IOException and HttpException.

3. Add an **Empty** state to UsersUiState for when the server returns an empty list.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why should screens not call Retrofit directly?

Q2. Why convert DTOs into app models?

Q3. Which exception means no internet or a timeout?

Q4. Which exception means the server returned an error code?

Q5. How can you test the error state quickly?

WEEK 10 • DAY 5

Images and Lists from the Network

Today's goal: Load images with Coil, build lists of network data, and send data with POST.

1. Coil: images from URLs

Dependency (Android only)

```
implementation("io.coil-kt.coil3:coil-compose:3.0.4")
implementation("io.coil-kt.coil3:coil-network-okhttp:3.0.4")   // Coil 3 needs a network module
```

AsyncImage

```
@Composable
fun Avatar(imageUrl: String, name: String) {
    AsyncImage(
        model = imageUrl,
        contentDescription = "Photo of $name",
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .size(56.dp)
            .clip(CircleShape)
            .background(MaterialTheme.colorScheme.surfaceVariant)   // shows while loading
    )
}
```

Coil feature	How
Loads the image in the background	Automatic
Caches images in memory and on disk	Automatic: scrolling back doesn't re-download
Placeholder / error image	placeholder = painterResource(R.drawable.loading), error = painterResource(R.drawable.broken)
Crops or fits	contentScale = ContentScale.Crop

2. A list with images

Photo list item

```
data class Photo(val id: Int, val title: String, val url: String)

@Composable
fun PhotoList(photos: List<Photo>, onPhotoClick: (Int) -> Unit) {
    LazyVerticalGrid(
        columns = GridCells.Adaptive(150.dp),
        contentPadding = PaddingValues(12.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(photos, key = { it.id }) { photo ->
            Card(onClick = { onPhotoClick(photo.id) }) {
                AsyncImage(
                    model = photo.url,
                    contentDescription = photo.title,
                    contentScale = ContentScale.Crop,
                    modifier = Modifier.fillMaxWidth().aspectRatio(1f)
                )
                Text(photo.title, maxLines = 2, overflow = TextOverflow.Ellipsis,
                    style = MaterialTheme.typography.bodySmall, modifier = Modifier.padding(8.dp))
            }
        }
    }
}
```

Tip: Practice image URLs: **https://picsum.photos/id/10/300/300** (change the id). The Coil library handles the rest.

3. Sending data with POST

Creating a post

```
@Serializable
data class NewPostRequest(val userId: Int, val title: String, val body: String)

@Serializable
data class CreatedPostResponse(val id: Int, val title: String)

interface PostApi {
    @POST("posts")
    suspend fun createPost(@Body post: NewPostRequest): CreatedPostResponse
}

class PostRepository(private val api: PostApi) {
    suspend fun publish(title: String, body: String): Int =
        api.createPost(NewPostRequest(userId = 1, title = title, body = body)).id
}
```

@Body tells Retrofit to encode the object as JSON and send it in the request body. JSONPlaceholder pretends to save it and answers with the new post (id 101), but nothing is really stored. That's fine for practice.

4. Response<T>: when you need the status code

Checking the response yourself

```
@Serializable
data class PostDto(val id: Int, val title: String)

interface PostApi {
    @GET("posts/{id}")
    suspend fun getPost(@Path("id") id: Int): Response<PostDto>
}

suspend fun loadTitle(api: PostApi, id: Int): String {
    val response = api.getPost(id)
    return when {
        response.isSuccessful -> response.body()?.title ?: "Empty response"
        response.code() == 404 -> "Post not found"
        else -> "Server error ${response.code()}"
    }
}
```

With **Response<T>**, Retrofit doesn't throw HttpException for error codes. You check **isSuccessful** and **code()** yourself.

5. Headers and authentication (a quick look)

Sending a token

```
@Serializable
data class ProfileDto(val name: String)

interface AccountApi {
    @GET("me")
    suspend fun getProfile(@Header("Authorization") token: String): ProfileDto
}
// call: api.getProfile("Bearer $accessToken")
```

Real apps usually add the token for **every** request with an OkHttp **Interceptor**, so you don't pass it in each function. Your team's project will already have this set up.

> **Careful:** Never put secret API keys or passwords directly in your code or in Git. Teams keep keys in local.properties, build configuration or on the server.

Practice Exercises

1. Build a **PhotoList** of 12 photos using picsum.photos URLs (ids 10 to 21).

2. Add **createTodo(@Body todo)** to an API interface for POST todos, with a request and response DTO.

3. Write a function that uses Response<UserDto> and returns "User not found" for 404.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Which composable loads an image from a URL with Coil?

Q2. Does Coil download the same image again when you scroll back?

Q3. What does @Body do?

Q4. When would you use Response?

Q5. Where should secret API keys not go?

WEEK 10 • DAY 6

Weekly Project: Weather App

Today's goal: Build a weather app that calls the free Open-Meteo API with Retrofit and shows loading, error and success states.

1. The API

Open-Meteo is free and needs no API key. Try this in your browser:

Sketch: Request

```
https://api.open-meteo.com/v1/forecast?latitude=9.93&longitude=76.26
    &current=temperature_2m,wind_speed_10m,weather_code&timezone=auto
```

The part of the response we need

```
{
  "latitude": 9.875,
  "longitude": 76.25,
  "timezone": "Asia/Kolkata",
  "current": {
    "time": "2026-10-16T12:00",
    "interval": 900,
    "temperature_2m": 30.2,
    "wind_speed_10m": 9.4,
    "weather_code": 2
  }
}
```

2. DTOs, mapping and weather codes (run with real JSON)

WeatherModels.kt

```
import kotlinx.serialization.SerialName
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

fun weatherDescription(code: Int): String = when (code) {
    0 -> "Clear sky"
    1, 2 -> "Partly cloudy"
    3 -> "Overcast"
    45, 48 -> "Fog"
    in 51..57 -> "Drizzle"
    in 61..67 -> "Rain"
    in 80..82 -> "Rain showers"
    in 95..99 -> "Thunderstorm"
    else -> "Unknown"
}
@Serializable
data class ForecastResponse(val current: CurrentDto)

@Serializable
data class CurrentDto(
    val time: String,
    @SerialName("temperature_2m") val temperature: Double,
    @SerialName("wind_speed_10m") val windSpeed: Double,
    @SerialName("weather_code") val weatherCode: Int
```

```
)

data class Weather(
    val city: String,
    val temperature: Double,
    val windSpeed: Double,
    val description: String
)

fun ForecastResponse.toWeather(city: String) = Weather(
    city = city,
    temperature = current.temperature,
    windSpeed = current.windSpeed,
    description = weatherDescription(current.weatherCode)
)

fun main() {
    val json = Json { ignoreUnknownKeys = true }
    val response = """{"latitude": 9.875, "longitude": 76.25, "timezone": "Asia/Kolkata",
        "current": {"time": "2026-10-16T12:00", "interval": 900, "temperature_2m": 30.2,
                    "wind_speed_10m": 9.4, "weather_code": 2}}"""
    val weather = json.decodeFromString<ForecastResponse>(response).toWeather("Kochi")
    println(weather)
    println(weatherDescription(63))
    println(weatherDescription(95))
}
```

Output

```
Weather(city=Kochi, temperature=30.2, windSpeed=9.4, description=Partly cloudy)
Rain
Thunderstorm
```

Open-Meteo uses the international **WMO weather codes** (0 = clear sky, 61-67 = rain, 95-99 = thunderstorm). The weatherDescription function turns them into words.

3. API, repository and network setup

WeatherApi.kt

```
interface WeatherApi {
    @GET("v1/forecast")
    suspend fun getCurrent(
        @Query("latitude") latitude: Double,
        @Query("longitude") longitude: Double,
        @Query("current") current: String = "temperature_2m,wind_speed_10m,weather_code",
        @Query("timezone") timezone: String = "auto"
    ): ForecastResponse
}

data class City(val name: String, val latitude: Double, val longitude: Double)

val keralaCities = listOf(
    City("Kochi", 9.93, 76.26),
    City("Thiruvananthapuram", 8.52, 76.94),
    City("Kozhikode", 11.26, 75.78),
    City("Munnar", 10.09, 77.06)
)

class WeatherRepository(private val api: WeatherApi) {
    suspend fun getWeather(city: City): Weather =
        api.getCurrent(city.latitude, city.longitude).toWeather(city.name)
}

object WeatherNetwork {
    private val json = Json { ignoreUnknownKeys = true }
    val api: WeatherApi = Retrofit.Builder()
        .baseUrl("https://api.open-meteo.com/")
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()
        .create(WeatherApi::class.java)
}
```

4. ViewModel

WeatherViewModel.kt

```
data class WeatherUiState(
    val selectedCity: City = keralaCities.first(),
    val weather: Weather? = null,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)

class WeatherViewModel(
    private val repository: WeatherRepository = WeatherRepository(WeatherNetwork.api)
) : ViewModel() {
    private val _uiState = MutableStateFlow(WeatherUiState())
    val uiState: StateFlow<WeatherUiState> = _uiState.asStateFlow()

    init { refresh() }

    fun onCitySelected(city: City) {
        _uiState.update { it.copy(selectedCity = city) }
        refresh()
    }

    fun refresh() {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                val weather = repository.getWeather(_uiState.value.selectedCity)
                _uiState.update { it.copy(weather = weather, isLoading = false) }
            } catch (e: IOException) {
                _uiState.update { it.copy(isLoading = false, errorMessage = "No internet connection") }
            } catch (e: HttpException) {
                _uiState.update { it.copy(isLoading = false, errorMessage = "Weather service error (${e.code()})") }
            }
        }
    }
}
```

5. Screen

WeatherScreen.kt

```
@Composable
fun WeatherRoute(viewModel: WeatherViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    WeatherScreen(uiState, onCitySelected = viewModel::onCitySelected, onRefresh = viewModel::refresh)
}

@Composable
fun WeatherScreen(uiState: WeatherUiState, onCitySelected: (City) -> Unit, onRefresh: () -> Unit) {
    Column(Modifier.fillMaxSize().padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        Text("Kerala Weather", style = MaterialTheme.typography.headlineMedium)

        LazyRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            items(keralaCities, key = { it.name }) { city ->
                FilterChip(
                    selected = city == uiState.selectedCity,
                    onClick = { onCitySelected(city) },
                    label = { Text(city.name) }
                )
            }
        }

        when {
            uiState.isLoading -> Box(Modifier.fillMaxWidth().height(200.dp), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
            uiState.errorMessage != null -> Column(horizontalAlignment = Alignment.CenterHorizontally,
                modifier = Modifier.fillMaxWidth()) {
                Text(uiState.errorMessage, color = MaterialTheme.colorScheme.error)
```

```
                TextButton(onClick = onRefresh) { Text("Retry") }
            }
            uiState.weather != null -> WeatherCard(uiState.weather)
        }
    }
}

@Composable
fun WeatherCard(weather: Weather) {
    ElevatedCard(modifier = Modifier.fillMaxWidth()) {
        Column(Modifier.padding(24.dp), horizontalAlignment = Alignment.CenterHorizontally) {
            Text(weather.city, style = MaterialTheme.typography.titleLarge)
            Text("${weather.temperature.roundToInt()} C", style = MaterialTheme.typography.displayLarge)
            Text(weather.description, style = MaterialTheme.typography.titleMedium)
            Spacer(Modifier.height(8.dp))
            Text("Wind: ${weather.windSpeed} km/h", style = MaterialTheme.typography.bodyMedium)
        }
    }
}

@Preview(showBackground = true)
@Composable
fun WeatherScreenPreview() {
    WeatherScreen(
        WeatherUiState(weather = Weather("Kochi", 30.2, 9.4, "Partly cloudy")),
        onCitySelected = { }, onRefresh = { }
    )
}
```

6. Run it

- Add the INTERNET permission, the Retrofit and serialization dependencies and the serialization plugin.
- Show **WeatherRoute()** in MainActivity's setContent.
- Tap the city chips: each one makes a real request. Watch the requests in Logcat if you added the logging interceptor.
- Turn on airplane mode and tap a city: you should see "No internet connection" and a Retry button.

7. Project challenges

- Add a 3-day forecast: request **daily=temperature_2m_max,temperature_2m_min** and show it in a row of cards.
- Show an icon for each weather description (e.g. Icons.Default.WbSunny from the extended icons).
- Remember the last selected city with rememberSaveable, or better, in the ViewModel's SavedStateHandle.
- Add a "Last updated" time from current.time.

Week 10 checklist

I can...	Done?
Explain HTTP methods, status codes and JSON	[ ]
Decode and encode JSON with kotlinx.serialization	[ ]
Handle snake_case, missing and unknown fields	[ ]
Define a Retrofit API with @GET, @Path, @Query and @Body	[ ]
Create a Retrofit instance with OkHttp logging	[ ]
Write a repository that maps DTOs to app models	[ ]
Handle IOException and HttpException in UiState	[ ]
Show network images with Coil	[ ]
Build the Weather app	[ ]

Quick Quiz

Q1. Why does the Weather app use @SerialName?

Q2. Where does the weather code become a description?

Q3. What happens when the user taps a city chip?

Q4. What does the user see in airplane mode?

Q5. Why is WeatherScreen easy to preview?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: How Apps Talk to Servers

Quiz answers

Q1. GET

Q2. Not Found: the resource doesn't exist.

Q3. 4xx means the app sent a bad request; 5xx means the server had a problem.

Q4. A List.

Q5. android.permission.INTERNET

Day 2: JSON with kotlinx.serialization

Exercise 1

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class Post(val userId: Int, val id: Int, val title: String, val body: String)

fun main() {
    val post = Json.decodeFromString<Post>("""{"userId": 1, "id": 7, "title": "magnam facilis", "body": "..."}""")
    println(post.title)
}
```

Output

```
magnam facilis
```

Exercise 2

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class Post(val userId: Int, val id: Int, val title: String)

fun main() {
    val posts = Json.decodeFromString<List<Post>>("""[
        {"userId": 1, "id": 1, "title": "a"},
        {"userId": 1, "id": 2, "title": "b"},
        {"userId": 2, "id": 3, "title": "c"}]""")
    posts.groupBy { it.userId }.forEach { (user, list) -> println("User $user: ${list.size} posts") }
}
```

Output

```
User 1: 2 posts
User 2: 1 posts
```

Exercise 3

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
import kotlinx.serialization.encodeToString

@Serializable
data class Todo(val userId: Int, val title: String, val completed: Boolean = false)

fun main() {
    println(Json.encodeToString(Todo(1, "Finish Week 10")))
}
```

Output

```
{"userId":1,"title":"Finish Week 10"}
```

Quiz answers

Q1. It makes the serialization plugin generate JSON conversion code for the class.

Q2. Json.decodeFromString(text)

Q3. So extra fields from the server don't crash decoding.

Q4. @SerialName("temperature_2m") val temperature: Double

Q5. Give the property a default value (and make it nullable if it can be null).

Day 3: Retrofit

Exercise 1

```
@Serializable data class UserDto(val id: Int, val name: String)

interface UserApi {
    @GET("users")
    suspend fun getUsers(): List<UserDto>

    @GET("users/{id}")
    suspend fun getUser(@Path("id") id: Int): UserDto
}
```

Exercise 2

```
@Serializable data class CommentDto(val postId: Int, val id: Int, val name: String, val body: String)

interface CommentApi {
    @GET("comments")
    suspend fun getComments(@Query("postId") postId: Int): List<CommentDto>
}
```

Exercise 3

```
@Serializable data class UserDto(val id: Int, val name: String)
interface UserApi { @GET("users") suspend fun getUsers(): List<UserDto> }

class UserNamesViewModel(private val api: UserApi) : ViewModel() {
    private val _names = MutableStateFlow<List<String>>(emptyList())
    val names: StateFlow<List<String>> = _names.asStateFlow()

    init {
        viewModelScope.launch {
            _names.value = try { api.getUsers().take(5).map { it.name } } catch (e: IOException) { emptyList() }
        }
    }
}
```

Quiz answers

Q1. As a Kotlin interface with annotated suspend functions.

Q2. @Path fills a {placeholder} in the path; @Query adds ?key=value to the URL.

Q3. A slash (/).

Q4. A Data Transfer Object: a class that matches the server's JSON exactly.

Q5. To see requests and responses in Logcat while debugging.

Day 4: Repository, Errors & UiState

Exercise 1

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class PostDto(val userId: Int, val id: Int, val title: String, val body: String)

data class Post(val id: Int, val title: String, val preview: String)

fun PostDto.toPost() = Post(
    id = id,
    title = title.replaceFirstChar { it.uppercase() },
    preview = body.take(100)
)

fun main() {
    val dto = Json.decodeFromString<PostDto>("""{"userId": 1, "id": 1, "title": "sunt aut facere",
        "body": "quia et suscipit suscipit recusandae"}""")
    println(dto.toPost())
}
```

Output

```
Post(id=1, title=Sunt aut facere, preview=quia et suscipit suscipit recusandae)
```

Exercise 2

```
@Serializable data class PostDto(val id: Int, val title: String)
data class Post(val id: Int, val title: String)
interface PostApi { @GET("posts") suspend fun getPosts(): List<PostDto> }

class PostRepository(private val api: PostApi) {
    suspend fun getPosts() = api.getPosts().map { Post(it.id, it.title) }
}

sealed interface PostsUiState {
    data object Loading : PostsUiState
    data class Success(val posts: List<Post>) : PostsUiState
    data class Error(val message: String) : PostsUiState
}

class PostsViewModel(private val repo: PostRepository) : ViewModel() {
    private val _uiState = MutableStateFlow<PostsUiState>(PostsUiState.Loading)
    val uiState: StateFlow<PostsUiState> = _uiState.asStateFlow()

    fun load() {
        viewModelScope.launch {
            _uiState.value = PostsUiState.Loading
            _uiState.value = try {
                PostsUiState.Success(repo.getPosts())
            } catch (e: IOException) {
                PostsUiState.Error("No internet connection")
            } catch (e: HttpException) {
                PostsUiState.Error("Server error ${e.code()}")
            }
        }
    }
}
```

Exercise 3

```
data class User(val id: Int, val name: String)

sealed interface UsersUiState {
    data object Loading : UsersUiState
    data object Empty : UsersUiState
    data class Success(val users: List<User>) : UsersUiState
    data class Error(val message: String) : UsersUiState
}

fun toUiState(users: List<User>): UsersUiState =
    if (users.isEmpty()) UsersUiState.Empty else UsersUiState.Success(users)
```

Quiz answers

Q1. Separation of concerns: the repository handles data and mapping; the ViewModel handles state; the screen only shows it.

Q2. The app is protected from server changes, and models contain exactly what the app needs.

Q3. IOException (for example UnknownHostException).

Q4. HttpException

Q5. Turn on airplane mode and open the screen.

Day 5: Images and Lists from the Network

Exercise 1

```
data class Photo(val id: Int, val title: String, val url: String)

val samplePhotos = (10..21).map { Photo(it, "Photo $it", "https://picsum.photos/id/$it/300/300") }

@Composable
fun SamplePhotoGrid() {
    LazyVerticalGrid(columns = GridCells.Fixed(3)) {
        items(samplePhotos, key = { it.id }) { photo ->
            AsyncImage(model = photo.url, contentDescription = photo.title,
                contentScale = ContentScale.Crop, modifier = Modifier.aspectRatio(1f))
        }
    }
}
```

Exercise 2

```
@Serializable data class NewTodoRequest(val userId: Int, val title: String, val completed: Boolean = false)
@Serializable data class TodoResponse(val id: Int, val title: String)

interface TodoApi {
    @POST("todos")
    suspend fun createTodo(@Body todo: NewTodoRequest): TodoResponse
}
```

Exercise 3

```
@Serializable data class UserDto(val id: Int, val name: String)

interface UserApi {
    @GET("users/{id}")
    suspend fun getUser(@Path("id") id: Int): Response<UserDto>
}

suspend fun userName(api: UserApi, id: Int): String {
    val response = api.getUser(id)
    return when {
        response.isSuccessful -> response.body()?.name ?: "Empty"
        response.code() == 404 -> "User not found"
        else -> "Error ${response.code()}"
    }
}
```

Quiz answers

Q1. AsyncImage

Q2. No, it caches images in memory and on disk.

Q3. Encodes the parameter as JSON and sends it as the request body.

Q4. When you need the status code or want to handle error codes without exceptions.

Q5. Directly in the code or in Git.

Day 6: Weekly Project: Weather App

Quiz answers

Q1. The JSON uses names like temperature_2m, but the Kotlin properties have clearer names.

Q2. In the mapping (toWeather) using weatherDescription, so the UI gets ready-to-show text.

Q3. The ViewModel updates selectedCity and calls refresh(), which loads the weather for that city.

Q4. "No internet connection" and a Retry button (from the IOException branch).

Q5. It's stateless, so a WeatherUiState with sample weather is enough.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 11** material. Great work, Anjana!